

CE4011 Structural Analysis GUI

Installation and Launch Manual

GUI-Based Static and Modal Postprocessing Extension of a 2D Frame-Truss Direct Stiffness Solver

Imran Shahriar - 2735835

19 June 2026

1. Purpose and recommended installation path

This manual explains how to install, launch, verify, and optionally rebuild the CE4011 static/modal structural analysis program. The software is a Python-based educational 2D frame-truss Direct Stiffness Method package with a Tkinter/Matplotlib graphical interface, static analysis, modal analysis, postprocessing, generated frame models, result tables, and selected linear modal response-history/response-spectrum utilities.

The program is intended for educational structural-analysis workflows. It is not a commercial structural design package.

Recommended for most users: Use Option A, the ZIP Windows executable distribution. Download, extract, and run the executable. No Conda setup is required for this route.

Recommended for developers: Use Option B, the Python/Conda source-code workflow. This is the most transparent and reproducible route for editing code, running tests, and regenerating examples.

2. Supported usage modes

Mode	Best for	Main command or action
Option A - ZIP Windows executable	Ordinary users who only want to run the GUI	Extract ZIP, then run the executable shown in Section 3.2.
Option B - Python/Conda source workflow	Users who want reproducibility, tests, scripts, and code access	conda activate CE4011, then python scripts\run_static_gui.py
Option C - Rebuild executable	Users who changed source code and need a fresh executable	.\build_exe.ps1 or python -m PyInstaller --clean --noconfirm CE4011_Solver.spec

3. Option A - ZIP Windows executable distribution

This is the quickest launch route. Use it when you only need to run the graphical application and do not plan to modify the source code.

3.1 Download sources for the ZIP distribution

Download the packaged Windows ZIP distribution from one of the following locations, depending on which package is available:

- GitHub repository or release page: <https://github.com/ImranMETU/CE4011-Structural-Analysis-GUI>
- Google Drive executable ZIP/package: https://drive.google.com/file/d/153He7D_wZ2FF9whGVxrezlJhlzXqVhss/

3.2 Extract and run

1. Download the ZIP package.
2. Extract the ZIP directly under C:\, or extract it to another folder where you have write permission.
3. After extraction, confirm that the executable and its _internal folder are still inside the same CE4011_Solver folder.
4. Run the executable:

C:\CE4011_Solver_Windows_x64\CE4011_Solver\CE4011_Solver.exe

Warning: Do not copy only CE4011_Solver.exe to the Desktop or another folder. The executable must remain next to its _internal folder. The _internal folder contains the embedded Python runtime, Tcl/Tk files, NumPy, SciPy, Matplotlib, and bundled demonstration data. Copying only the .exe will usually cause a "Failed to load Python DLL" error.

3.3 Expected startup behavior

The main window should open with a title similar to:

CE4011 Static/Modal Result Viewer

After launch, open a generated example such as model_a_5story_unbraced.json, run static analysis, and then run modal analysis if the companion mass file is available.

4. System requirements for the source-code workflow

- Windows 10 or Windows 11.
- Anaconda or Miniconda.
- Python 3.11.
- A working Tkinter installation.
- Python packages normally installed from requirements.txt.

Core package categories are:

- numpy for numerical arrays.
- scipy for eigenvalue and sparse/numerical routines.
- matplotlib for plots and embedded GUI figures.
- pytest for verification tests.
- Python standard-library modules including tkinter, json, xml, csv, pathlib, and sys.

5. Option B - Python/Conda source-code workflow

5.1 Clone or download the repository

Clone the repository or download the source ZIP from GitHub:

```
git clone https://github.com/ImranMETU/CE4011-Structural-Analysis-GUI.git
cd CE4011-Structural-Analysis-GUI
```

If the repository was downloaded as a ZIP, extract it and open PowerShell or Anaconda Prompt in the repository root.

5.2 Repository root requirement

Run source-code commands from the repository root. Running scripts from another folder may break relative paths for inputs, generated models, verification files, results, or companion modal mass files.

```
cd "path\to\CE4011-Structural-Analysis-GUI"
```

Example development path used during the project:

```
cd "C:\DDrive\Imran\METU\Coursework\CE4011\Project"
```

5.3 Create and activate the Conda environment

If the CE4011 environment already exists, activate it and skip the creation command. Otherwise, create a Python 3.11 environment:

```
conda create -n CE4011 python=3.11 -y
conda activate CE4011
```

5.4 Install dependencies

Install dependencies from the repository root:

```
python -m pip install -r requirements.txt
```

If requirements.txt is not available, install the essential packages manually:

```
python -m pip install numpy scipy matplotlib pytest
```

Tkinter usually comes with the standard Python distribution on Windows/Anaconda. If the GUI does not launch and the error mentions tkinter, install a Python distribution that includes Tk support.

5.5 Launch the GUI from source

From the repository root, activate the environment and run the launcher:

```
conda activate CE4011
python scripts\run_static_gui.py
```

If the environment is not activated but its Python executable is known, the launcher may also be run with an absolute Python path. Example:

```
C:\Users\Imran\anaconda3\envs\CE4011\python.exe scripts\run_static_gui.py
```

The main window should open with a title similar to:

CE4011 Static/Modal Result Viewer

6. Option C - Rebuilding the executable from source

A packaged executable can be rebuilt from the repository root if PyInstaller and the CE4011 environment are available. Use the provided build script when available:

```
.\build_exe.ps1
```

Equivalent manual command:

```
conda activate CE4011
python -m PyInstaller --clean --noconfirm CE4011_Solver.spec
```

The rebuilt executable should appear under:

```
dist\CE4011_Solver\CE4011_Solver.exe
```

Warning: The build\ directory is temporary PyInstaller working output. It is not the application distribution folder and should not be used to launch the program. For distribution, ZIP the complete dist\CE4011_Solver folder.

7. Basic verification commands

Run these commands from the repository root after installation or before handing the project to another user:

```
python -m pytest -q
python scripts\run_gui_backend_verification.py
python scripts\run_final_core_smoke.py
```

Expected results for the cleaned final project version:

- Pytest suite: 344 tests passed.
- GUI backend verification: Total 13, PASS 13, FAIL 0.
- Final core smoke checks: all checks PASS.

A Windows temporary-directory cleanup warning after pytest may appear. If the tests themselves pass, this warning is not treated as a solver failure.

8. Optional verification and example scripts

The following scripts generate example result files and figures under results/ where implemented:

```
python scripts\generate_proposal_models.py
python scripts\run_member_load_examples.py
python scripts\run_station_postprocessing_case.py
python scripts\run_solver_diagnostics_case.py
python scripts\compare_braced_unbraced.py
python scripts\run_modal_rha_case.py
python scripts\run_modal_rsa_case.py
```

The response-history and response-spectrum scripts are supplementary linear modal postprocessing examples. They are not nonlinear seismic design workflows.

9. Generated outputs

Most generated outputs are written under results/. Common output folders may include:

- results/final_core_smoke/
- results/member_load_examples/
- results/station_postprocessing/
- results/solver_diagnostics/
- results/comparisons/
- results/rha/

Generated outputs can be regenerated from the scripts and are normally treated as analysis products rather than source files.

10. Input files and companion modal mass files

The program supports JSON, XML, text-deck, form-based GUI input, and generated parametric frame models. Modal masses may be stored in the main model file or in a companion mass file. Example:

```
model_a_5story_unbraced.json
model_a_5story_unbraced_masses.json
```

When opening generated or saved modal models, ensure that the companion mass file remains in the expected location if the model depends on explicit modal masses.

11. Troubleshooting

Problem	Recommended action
Executable reports Failed to load Python DLL	Run the executable from inside the complete CE4011_Solver folder. Do not move the .exe away from _internal. Re-extract the ZIP if necessary.
GUI does not launch from source	Check that you are running from the repository root and using the CE4011 Python environment. Confirm that Tkinter is available.
Import error or circular import message	Run from the repository root, not from inside src/. Then run: <code>python -m pytest tests\test_import_stability.py -q</code>
Model opens but modal analysis fails	Check that the model has modal mass data. If a companion mass file is missing, restore it or define masses through the GUI.
Static deformed shape looks undeformed	The numerical displacement may be small. Use the autoscaled deformed-shape plot or increase the display scale. Read actual displacement values from the tables.
Force diagram sign differs from Ftool	This is usually a display convention issue. Numerical internal-force values are separate from the plotted side/sign convention.
Units appear wrong	The unit system is metadata and labeling only. Changing displayed unit labels does not rescale existing numerical input values.

12. Software scope reminder

The software is a linear elastic 2D frame-truss educational analysis package. It does not perform code design checks, nonlinear response-history analysis, full 3D frame analysis, shell/slab modeling, or rigid diaphragm modeling.